BUILDING A RASPBERRY PI SYSTEM MONITORING STATION

A Step-by-Step Guide to Real-Time Performance Analysis

PROJECT GOAL

Build a tool that records CPU, RAM, disk usage, and temperature.

UNDERSTANDING SYSTEM MONITORING IN EMBEDDED ENGINEERING

THE CONCEPT

System monitoring is the continuous process of tracking hardware performance metrics to ensure stability and efficiency.

It involves collecting data from various system components and presenting it in a way that is actionable for engineers.

WHY IT MATTERS

- → Detecting performance bottlenecks
- → Preventing hardware overheating
- → Optimizing resource allocation
- → Ensuring long-term system health

STEP 1: GATHERING HARDWARE COMPONENTS

Before starting, ensure you have all the necessary hardware components. Each part plays a vital role in the system's operation.

COMPONENT	PURPOSE
Raspberry Pi 4	The central processing unit for data collection and analysis.
MicroSD Card (16GB+)	Houses the OS, your Python scripts, and the logged data.
Power Supply (5V 3A)	Ensures stable operation, especially under high CPU load.
Peripherals	Monitor, keyboard, and mouse for initial setup and coding.

STEP 2: PREPARING THE OPERATING SYSTEM

INSTALLING RASPBERRY PI OS WITH THE IMAGER TOOL

01 | **DOWNLOAD & LAUNCH:** Obtain and open the official Raspberry Pi Imager on your primary computer.

02 | **SELECT OS:** Choose **RASPBERRY PI OS (64-BIT)** for modern library support and performance.

03 | **SELECT STORAGE:** Insert your MicroSD card (16GB+) and select it as the target drive in the tool.

04 | **WRITE & VERIFY:** Click 'Write'. Once finished, insert the card into your Pi and boot it up.

STEP 3: INITIAL SYSTEM CONFIGURATION

Once booted, the Linux Terminal is your primary interface for system management. It allows you to interact directly with the OS kernel to install software and manage resources.

TERMINAL SHORTCUT: CTRL + ALT + T

UPDATING SYSTEM PACKAGES

```
sudo apt update
sudo apt upgrade -y
```

- **update:** Refreshes the local database of available software packages from the official repositories.
- **upgrade:** Installs the latest versions of all currently installed software, ensuring stability and security.

STEP 4: SETTING UP THE PYTHON ENVIRONMENT

INSTALLING ESSENTIAL LIBRARIES

Our monitoring tool relies on three key Python libraries. We will use `pip`, the Python package installer, to set them up.

- **psutil:** Retrieves system-level information including CPU, RAM, and Disk utilization.
- **matplotlib:** A comprehensive library for creating static and interactive visualizations.
- **pandas:** Provides high-performance data structures and data analysis tools.

LINUX TERMINAL - PACKAGE INSTALLATION

```
pi@raspberrypi:~ $ pip3 install psutil
matplotlib pandas
```

STEP 5: CODING THE CORE MONITOR

REAL-TIME METRIC COLLECTION

Create a file named `monitor.py`. This script will use the `psutil` library to fetch system data every few seconds.

- **Continuous Loop**: The `while True` loop ensures the system is monitored indefinitely.
- **Sampling Interval**: `psutil.cpu_percent(interval=1)` blocks for 1 second to calculate an accurate average.
- **Live Feedback**: Data is printed directly to the terminal for immediate observation.

```python
import psutil
import time

def get_metrics():
    # Fetch system statistics
    cpu = psutil.cpu_percent(interval=1)
    ram = psutil.virtual_memory().percent
    disk = psutil.disk_usage('/').percent

    # Output to terminal
    print(f"CPU: {cpu}% | RAM: {ram}% | Disk: {disk}%")

if __name__ == "__main__":
    while True:
        get_metrics()
        time.sleep(2)
```

STEP 6: ACCESSING HARDWARE TEMPERATURE

INTERFACING WITH SENSORS

Monitoring temperature is vital to prevent thermal throttling. We use the `vcgencmd` utility via Python's `subprocess` module.

- **vcgencmd**: A Raspberry Pi specific tool for low-level hardware access.
- **measure_temp**: The specific command to retrieve SoC temperature.
- **Data Parsing**: The output is a string (e.g., 'temp=45.0\'C') that must be decoded and cleaned.

```python
import subprocess

def get_temp():
    # Execute system command
    raw = subprocess.check_output(
        ['vcgencmd', 'measure_temp']
    )

    # Decode 'temp=45.0\'C' to 45.0
    temp_str = raw.decode()
    temp_val = temp_str.split('=')[1]
    temp_val = temp_val.split("'")[0]

    return float(temp_val)

# Example usage
print(f"Current Temp: {get_temp()}°C")
```

STEP 7: IMPLEMENTING THE DATA LOGGER

SAVING PERFORMANCE HISTORY

To analyze data over time, we must save it. We'll create a logger that appends metrics to a `system_log.csv` file.

- **CSV Format**: A simple, comma-separated format that is easy to read with both Python and spreadsheet software.
- **Append Mode**: Using `'a'` ensures that new data is added to the end of the file without overwriting existing records.
- **Persistence**: Logging transforms real-time observations into a historical dataset for long-term analysis.

```python
import csv
import time

def log_metrics(cpu, ram, temp):
    # Open file in append mode
    with open('system_log.csv', 'a', newline='') as f:
        writer = csv.writer(f)

        # Record: Timestamp, CPU, RAM, Temp
        writer.writerow([
            time.time(),
            cpu,
            ram,
            temp
        ])

# Call this inside your main loop
```

STEP 8: VISUALIZING THE RESULTS

GENERATING PERFORMANCE GRAPHS

The final step is to turn your log file into a visual report. This script reads the CSV and plots the CPU and RAM usage.

- **Data Loading**: Use `pandas` to read the CSV and assign column names.
- **Plotting**: Use `matplotlib` to create a line graph of system metrics.
- **Report Generation**: Save the plot as a PNG file for inclusion in engineering reports.

Visualizing data makes it much easier to spot spikes in resource usage or potential overheating trends.

```python
import matplotlib.pyplot as plt
import pandas as pd

def plot_data():
    # Load log file
    cols = ['Time', 'CPU', 'RAM', 'Temp']
    data = pd.read_csv('system_log.csv', names=cols)

    # Create plot
    plt.figure(figsize=(10, 5))
    plt.plot(data['CPU'], label='CPU %')
    plt.plot(data['RAM'], label='RAM %')

    # Formatting
    plt.legend()
    plt.title('System Performance Over Time')
    plt.ylabel('Utilization (%)')

    # Save result
    plt.savefig('report.png')

if __name__ == "__main__":
    plot_data()
```

TROUBLESHOOTING & BEST PRACTICES

PERMISSION ISSUES

Ensure your user has the necessary rights to execute `vcgencmd` and write to your project directory. Use `sudo` if necessary for system-level commands.

SAMPLING RATE

Avoid logging too frequently (e.g., every 0.1s). High-frequency monitoring can consume significant CPU resources, skewing your performance data.

DATA MANAGEMENT

Periodically clear or archive your log files. Continuous logging can eventually fill up your MicroSD card, leading to system instability.

CODE MODULARITY

Keep your collection, logging, and plotting functions separate. This modular approach makes it much easier to debug and expand your system.

CONCLUSION & NEXT STEPS

SKILLS MASTERED

- → Linux administration & package management
- → Hardware-software interfacing with Python
- → Data persistence using CSV logging
- → Performance visualization techniques

FUTURE EXPANSIONS

- → Add an OLED display for real-time stats
- → Implement email alerts for overheating
- → Develop a web-based monitoring dashboard
- → Integrate remote logging via MQTT

CONGRATULATIONS! YOU HAVE BUILT A FOUNDATIONAL TOOL FOR REAL-WORLD EMBEDDED SYSTEMS ENGINEERING.